Übung 6: Vererbung und Interfaces (Obligatorische Abgabe!)

Aufgabe 1: Overloading, Overriding & Dynamic Binding 1
Aufgabe 2: Vereinfachtes Dateisystem 3
Aufgabe 3: Gleichheit 3
Aufgabe 4: Overloading und Overriding (Theorie) 5
Aufgabe 5: Syntax von Schnittstellen (Einstiegsaufgabe) 6
Aufgabe 6: Mehrfachvererbung (fakultativ) 6

Lernziele

- Klassen, die in einer Vererbungshierarchie stehen, implementieren können.
- Methoden überschreiben können, um polymorphes Verhalten zu erzielen.
- Schnittstellen designen und implementieren.
- Mehrfach-Implementierung und Spezialisierung von Interfaces üben.

Hinweis:

- Beachten Sie, dass Sie diese Übung abgeben müssen, um zur Prüfung zugelassen zu werden! Sie finden die Frist für Ihre Übungsgruppe in Moodle.
- Aufgabe 4 (Theorie) müssen Sie nicht abgeben.

Aufgabe 1: Overloading, Overriding & Dynamic Binding

Gegeben sind folgende Klassen `Person` und `Student`. Die Klasse `Person` ist wie folgt implementiert:

```
1  class Person {
2
3      public void introduce() {
4          IO.println("I am a person.");
5      }
6
7      public void introduce(String greeting) {
8          IO.println(greeting + " I am a person.");
9      }
10 }
```

Die von der Klasse `Person` abgeleitete Klasse `Student` ist wie folgt implementiert:

```
1  class Student extends Person {
2
3      @Override
4      public void introduce() {
5          IO.println("I am a student.");
6      }
7
8      public void introduce(int semester) {
9          IO.println("I am in semester " + semester + ".");
10     }
11 }
```

Prüfen Sie zunächst eine Verwendung wie folgt:

```
1  public class StudentUsage {
2      void main() {
3
4          var student = new Student();
5
6          student.introduce();
7          student.introduce("Hello");
8          student.introduce(3);
9      }
10 }
```

Untersuchen Sie anschliessend:

```
1  Person person = new Student();
2
3  person.introduce();
4  person.introduce("Hello");
```

Versuchen Sie danach einmal diesen Aufruf:

```
1  Person person =  new Student();
2
3  person.introduce(3);
```

Beantworten Sie folgende Fragen:

1. Welche Methoden sind überladen? Person.introduce, Student.introduce
2. Welche Methode wird überschrieben? Person.introduce() durch Student.introduce()
3. Welchen statischen Typ hat die Variable `person`? Person
4. Welchen dynamischen Typ hat die Variable `person`? Student
5. Welche Methode wird bei `person.introduce()` ausgeführt? Student.introduce()
6. Warum funktioniert `person.introduce(3)` nicht?
7. Welche Rolle spielen statischer und dynamischer Typ beim Methodenaufruf?

Aufgabe 2: Vereinfachtes Dateisystem

Implementieren Sie verschiedene Klassen, um Dateien (`File`), Verknüpfungen (`Link`) und Verzeichnisse (`Folder`) als Modellierung eines Dateisystems abzubilden. Diese Klassen sollen eine gemeinsame Basisklasse (`FilesystemElement`) besitzen, die jede Art von Dateisystemelement repräsentiert: Sprich eine Datei ist ein Dateisystemelement, eine Verknüpfung ist ein Dateisystemelement und ein Verzeichnis ist ebenfalls ein Dateisystemelement.

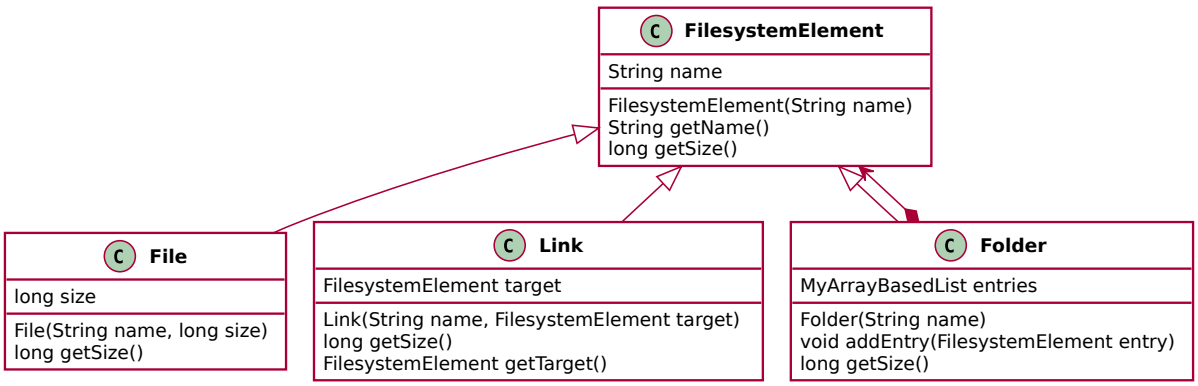


Figure 1: Klassendiagramm mit Vererbungshierarchie

Realisieren Sie die Klassen wie folgt:

- `FilesystemElement` ist die Basisklasse und bietet die Funktionalität, um auf den Namen zuzugreifen, welchen jedes Dateisystemelement hat. Diesen Namen speichert die Basisklasse selbst und entsprechend muss der Wert im Konstruktor übergeben werden. Zudem bietet die Basisklasse die Möglichkeit, die Grösse des Elements abzufragen. Diese soll aber nicht in der Basisklasse gespeichert, sondern von jeder abgeleiteten Klasse bereitgestellt werden.
- `File` repräsentiert eine Datei, welche eine gegebene Grösse aufweist.
- `Link` ist eine Verknüpfung. Die Grösse der Verknüpfung ist immer fix 4000. Zudem existiert einen Verweis auf ein anderes `FilesystemElement` als Ziel der Verknüpfung.
- `Folder` stellt ein Verzeichnis dar. Dieser kann weitere Dateisystemelemente beinhalten. Die Grösse eines Folders errechnet sich jeweils aus der Grösse der beinhalteten Elemente.

Aufgabe 3: Gleichheit

Ergänzend wollen wir für die obigen Klassen des Dateisystems noch die `equals()`-Methode anbieten. Bevor Sie diese in allen Klassen implementieren, überlegen Sie sich, für welche Klassen dies überhaupt sinnvoll ist.

ACHTUNG: Sie müssen für diese Übung *nicht* auch die `hashCode()`-Methode implementieren. In der Praxis muss jedoch, immer wenn die `equals()`-Methode überladen wird, stets auch die `hashCode()`-Methode überladen werden.

Aufgabe 4: Overloading und Overriding (Theorie)

Überlegen Sie sich die Fälle hier zuerst und überprüfen Sie diese dann mit einem laufenden Programm. Gegeben sind zwei Klassen:

```java
class Base {
    void copyTo(Object other) {
        // Method 1
    }

    void copyTo(Base other) {
        // Method 2   }
    }
}
```

```java
class Sub extends Base {
    void copyTo(Base other) {
        // Method 3
    }
    void copyTo(Sub other) {
        // Method 4
    }
}
```

Bestimmen Sie, welche Methode jeweils ausgeführt wird. Erkennen Sie auch, wo es Compiler-Fehler gibt und begründen Sie warum.

```java
Sub sub = new Sub();
Base base = new Sub();
Object obj = new Sub();
var v = obj;
```

Aufruf	Methode
base.copyTo(obj);	
base.copyTo(base);	
base.copyTo(sub);	
sub.copyTo(obj);	
sub.copyTo(base);	
sub.copyTo(sub);	
v.copyTo(v);	

Aufgabe 5: Syntax von Schnittstellen (Einstiegsaufgabe)

Um sich mit der Syntax von Schnittstellen vertraut zu machen, definieren Sie eine Schnittstelle `Logger`. Diese Schnittstelle soll eine Methode `void log(String message)` anbieten, welche die übergebene Nachricht loggt. Wo und wie dieses Logging geschieht spezifiziert die Schnittstelle selbst nicht.

Nun könnten verschiedene Klassen diese Schnittstelle implementieren. Folgende Klassen sind bereits vorgegeben, allerdings ohne die Schnittstelle zu implementieren:

- `NullLogger`, diese Klasse macht nichts in der `log()`-Methode
- `FileLogger`, diese Klasse speichert die Nachrichten in eine Datei, welche bei der Konstruktion mitgegeben wird.
- `ConsoleLogger`, diese Klasse schreibt die Nachricht auf die Konsole.

Passen Sie die Klassen so an, dass sie die Schnittstelle `Logger` implementieren. Schreiben Sie ein `main()`-Programm, welches jeden Logger einmal instanziiert und aufruft.

Aufgabe 6: Mehrfachvererbung (fakultativ)

In Java gibt es keine Mehrfachvererbung mit Klassen. Also wird folgendes Beispiel nicht unterstützt:

```
1  class A {
2      void f() { IO.println("A::f()"); }
3      void g() { IO.println("A::g()"); }
4  }
5
6  class B {
7      void h() { IO.println("B::h()"); }
8  }
9
10 class C extends A, B { } // impossible in Java
```

Wie könnte man mittels Interfaces die Mehrfachvererbung dennoch umsetzen? Konkret soll gelten:

- Typ `C` ist ein Sub-Typ von `A` und `B`.
- Typ `C` verwendet denselben Code von den Implementierungen von `A` und `B` wieder.

Der einzige Unterschied ist, dass Sie nicht `new A()` bzw. `new B()` aufrufen können, sondern die entsprechenden Klassen für `A` und `B` anders instanziieren müssen.

Zusatzfrage 1: Inwiefern sind hier Java 8 Default Methods nützlich?

Zusatzfrage 2: Klasse `A` und `B` sollten auch Instanzvariablen enthalten. Wie kann man auch dies umsetzen?